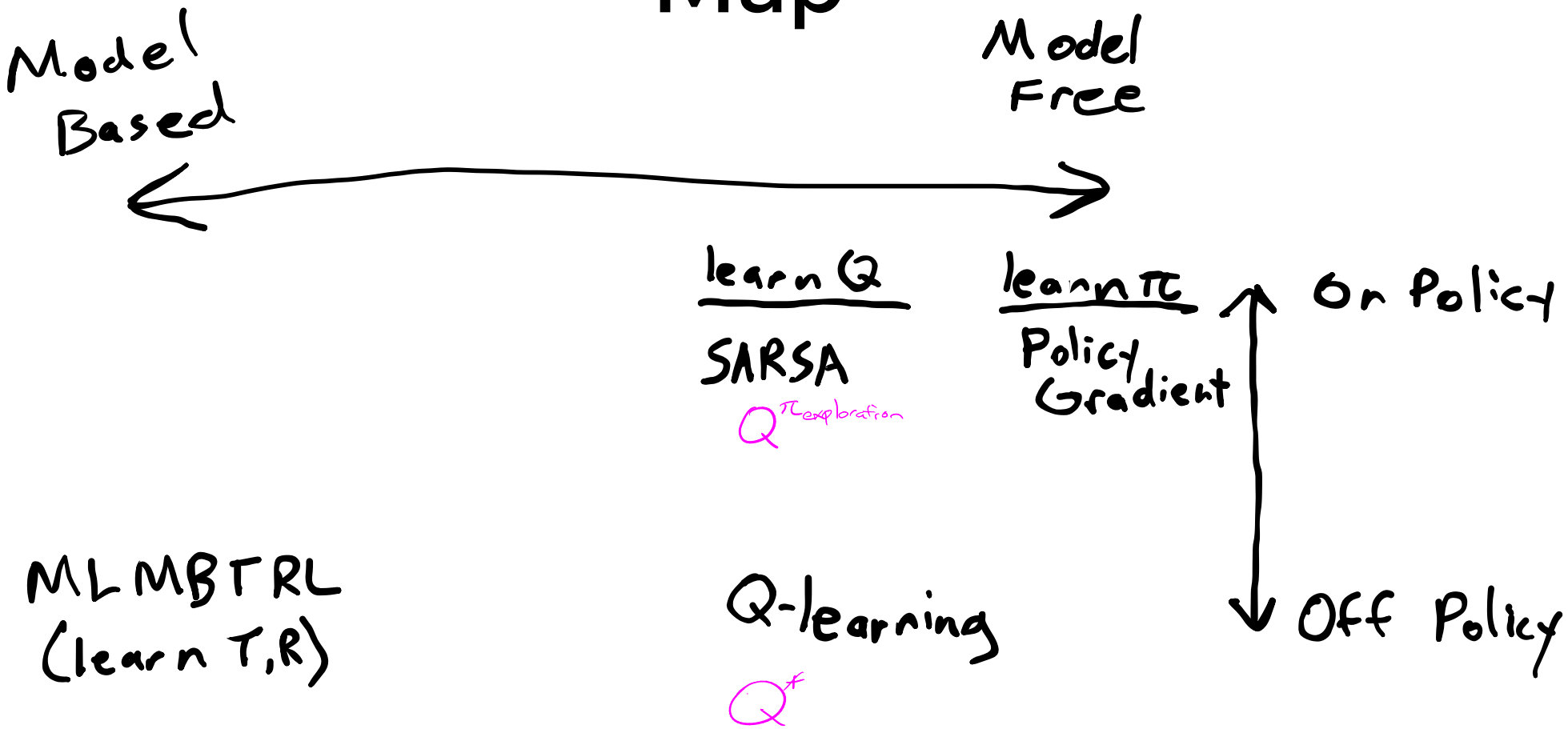
DQN and Advanced Policy Gradient

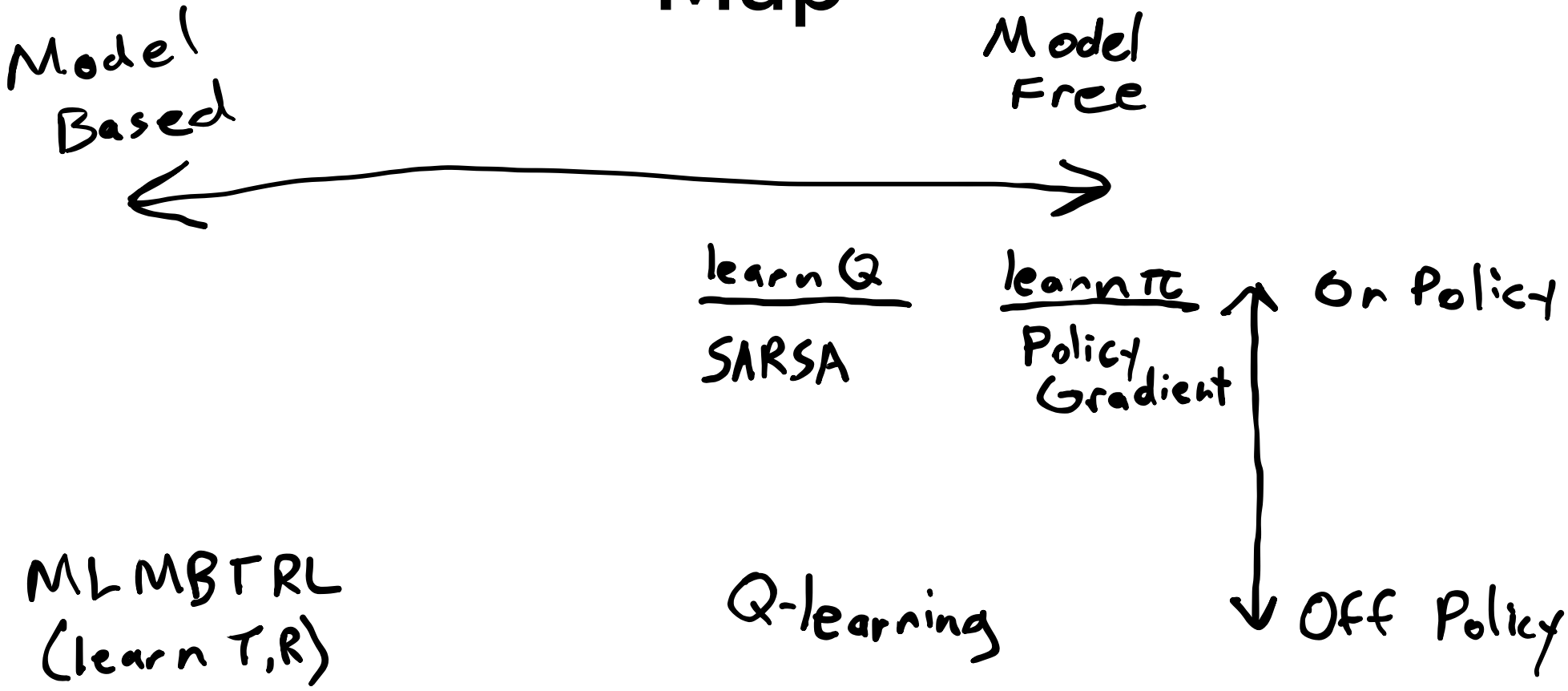
Map

Map

Map


Model Based
Model Free
learn Q
learn π
SARSA
Policy Gradient
On Policy
Off Policy
ML MBTRL (learn T,R)
Q-learning

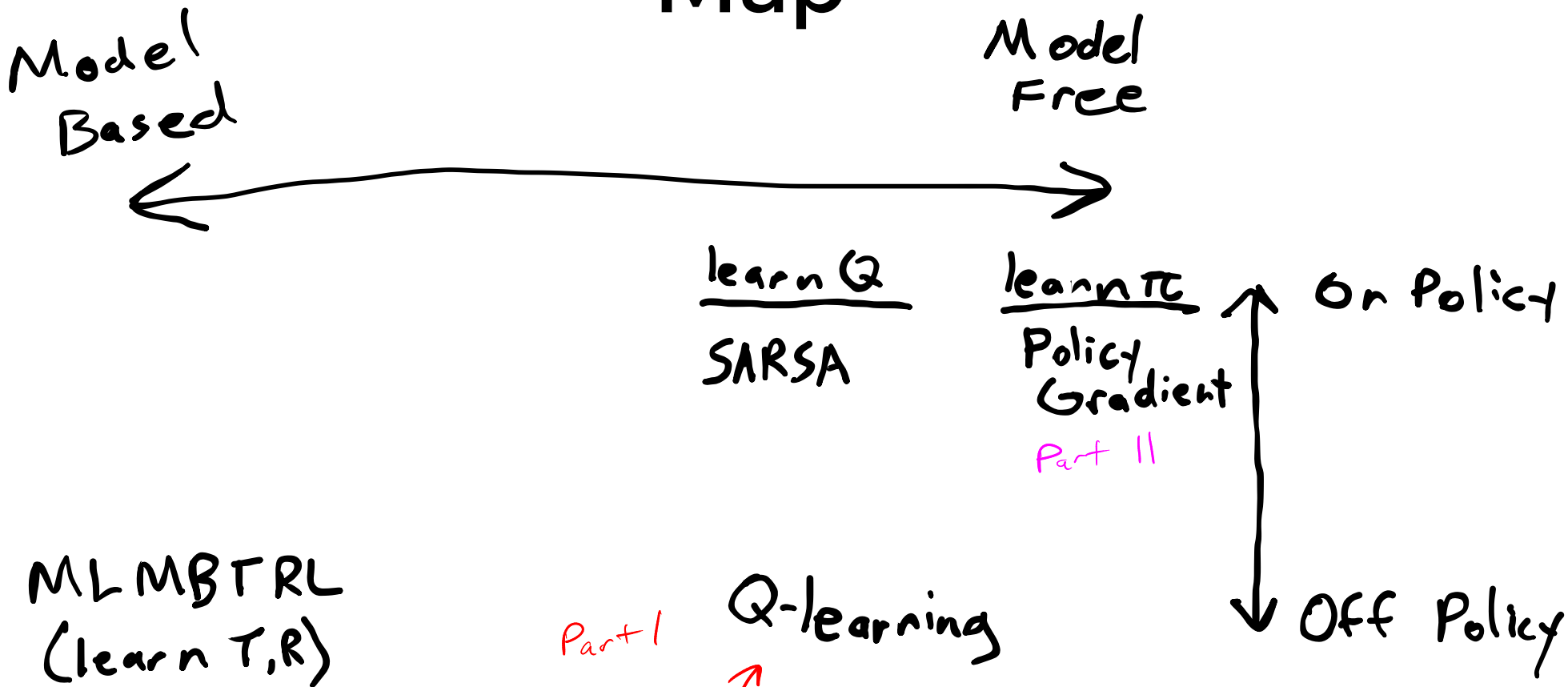
Map



Challenges:

1. Exploration vs Exploitation
2. Credit Assignment
3. Generalization

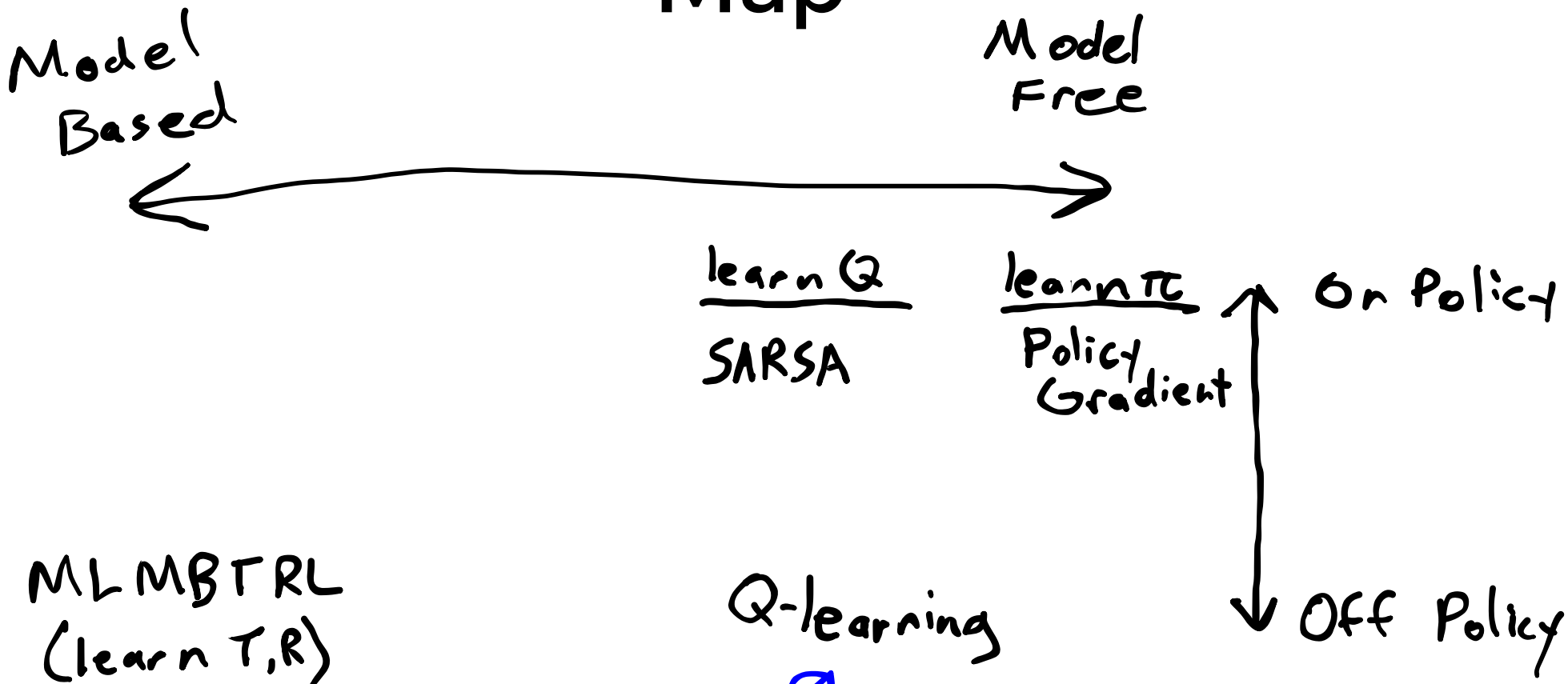
Map



Challenges:

1. Exploration vs Exploitation
2. Credit Assignment
3. Generalization

Map

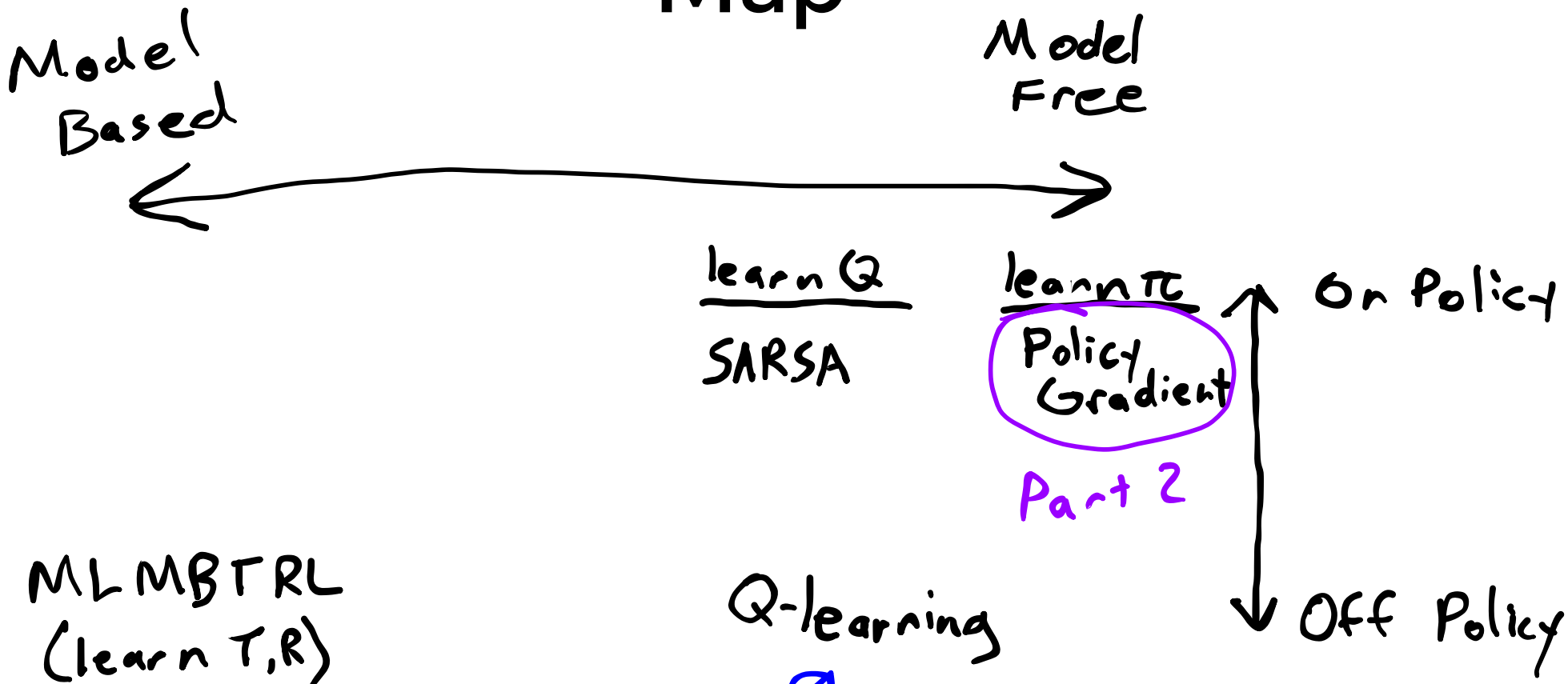


Challenges:

1. Exploration vs Exploitation
2. Credit Assignment
3. Generalization

Last Time: Neural Networks

Map

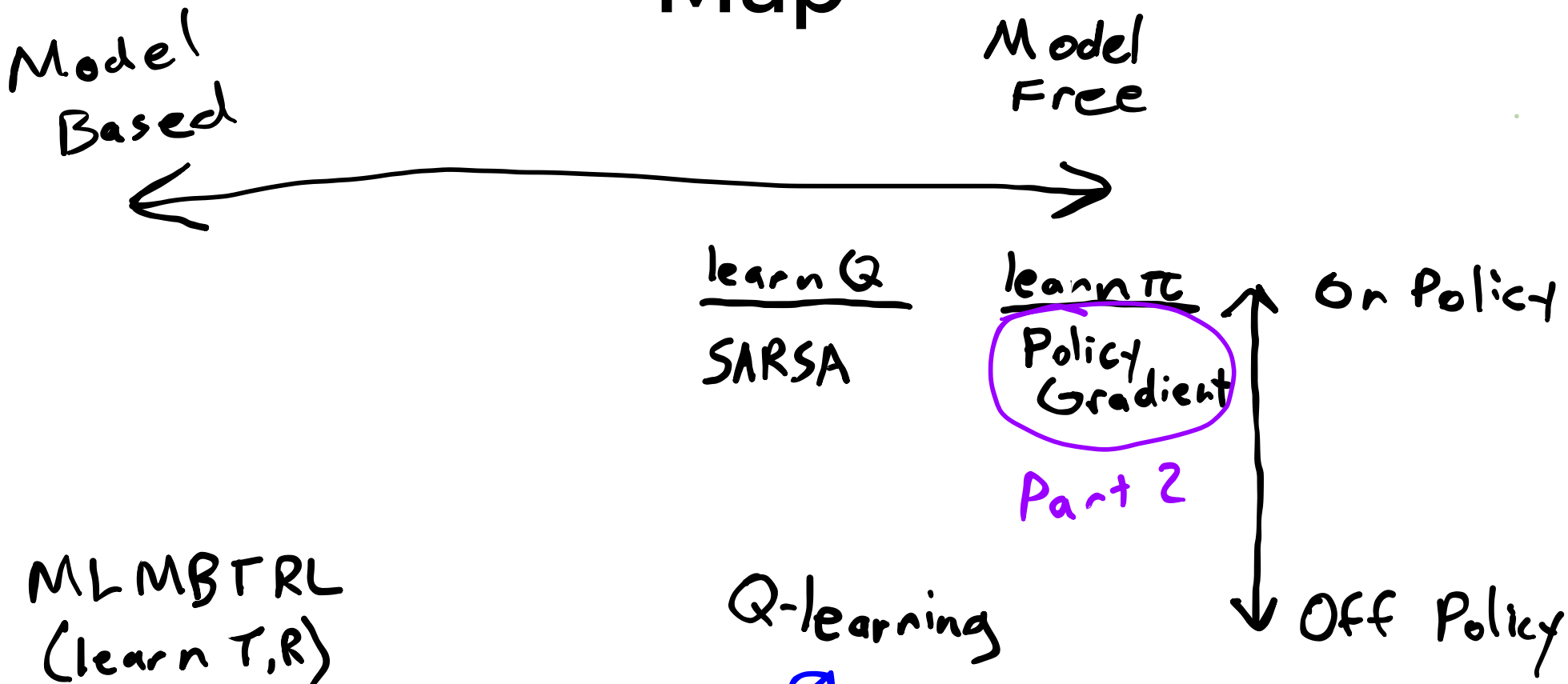


Challenges:

1. Exploration vs Exploitation
2. Credit Assignment
3. Generalization

Last Time: Neural Networks

Map



Challenges:

1. Exploration vs Exploitation
2. Credit Assignment
3. Generalization

Part I
DQN

Q-Learning with Neural Networks

Q-Learning with Neural Networks

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$$

Q-Learning with Neural Networks

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$$

Neural Networks

$$\theta^* = \arg\min_{\theta} \sum_{(x,y)\in\mathcal{D}} l(f_\theta(x), y)$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta l(f_\theta(x), y)$$

Q-Learning with Neural Networks

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Neural Networks

$$\theta^* = \arg\min_\theta \sum_{(x,y)\in\mathcal{D}} l(f_\theta(x), y)$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \, l(f_\theta(x), y)$$

Deep Q learning:

- Approximate Q with Q_θ

Q-Learning with Neural Networks

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Neural Networks

$$\theta^* = \arg\min_\theta \sum_{(x,y)\in\mathcal{D}} l(f_\theta(x), y)$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta\, l(f_\theta(x), y)$$

Deep Q learning:

- Approximate Q with Q_θ
- What should (x, y) be?

Q-Learning with Neural Networks

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Neural Networks

$$\theta^* = \arg\min_\theta \sum_{(x,y)\in\mathcal{D}} l(f_\theta(x), y)$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta\, l(f_\theta(x), y)$$

Deep Q learning:

- Approximate Q with Q_θ
- What should (x, y) be?
- What should l be?

Want

$$r + \gamma \max_{a'} Q_\theta(s',a') = Q_\theta(s,a)$$

Loss

$$l(s,a,r,s') = \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

Q-Learning with Neural Networks

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Neural Networks

$$\theta^* = \arg\min_\theta \sum_{(x,y)\in\mathcal{D}} l(f_\theta(x), y)$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta\, l(f_\theta(x), y)$$

Deep Q learning:

- Approximate Q with Q_θ
- What should (x, y) be?
- What should l be?

Candidate Algorithm:

loop

$$a \leftarrow \text{argmax}\, Q(s,a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$$

$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

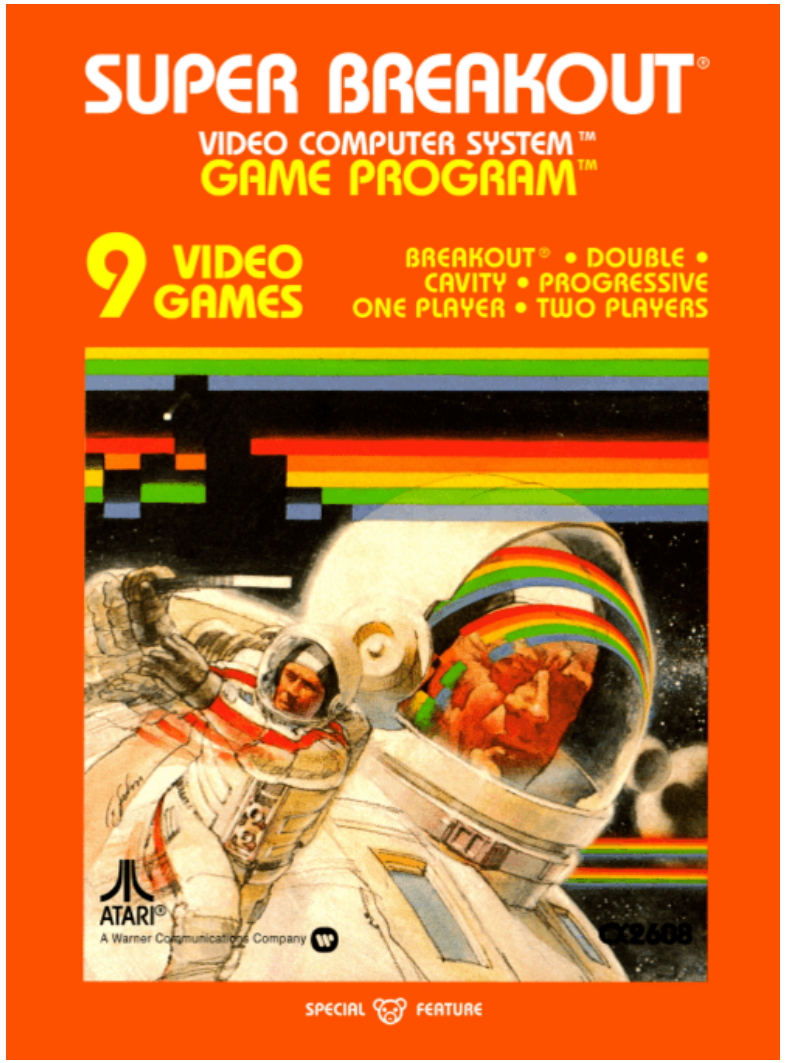
$$s \leftarrow s'$$

$\nabla_\theta\, l(s,a,r,s')$

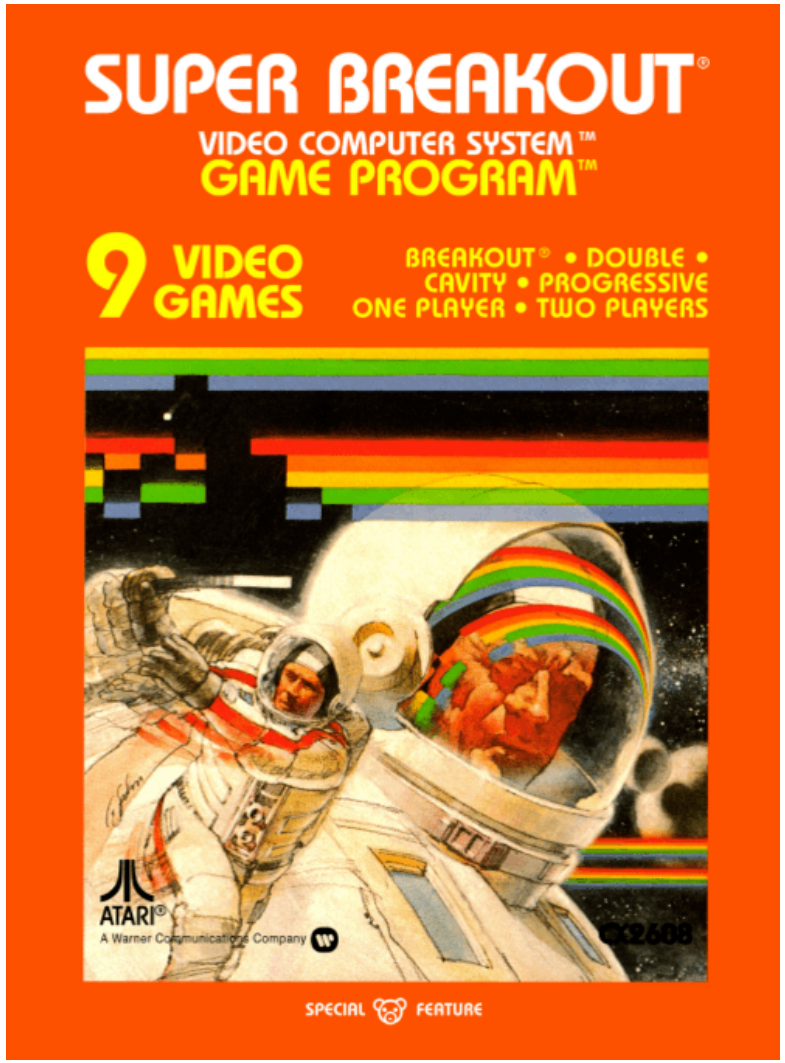
DQN: The Atari Benchmark

DQN: The Atari Benchmark

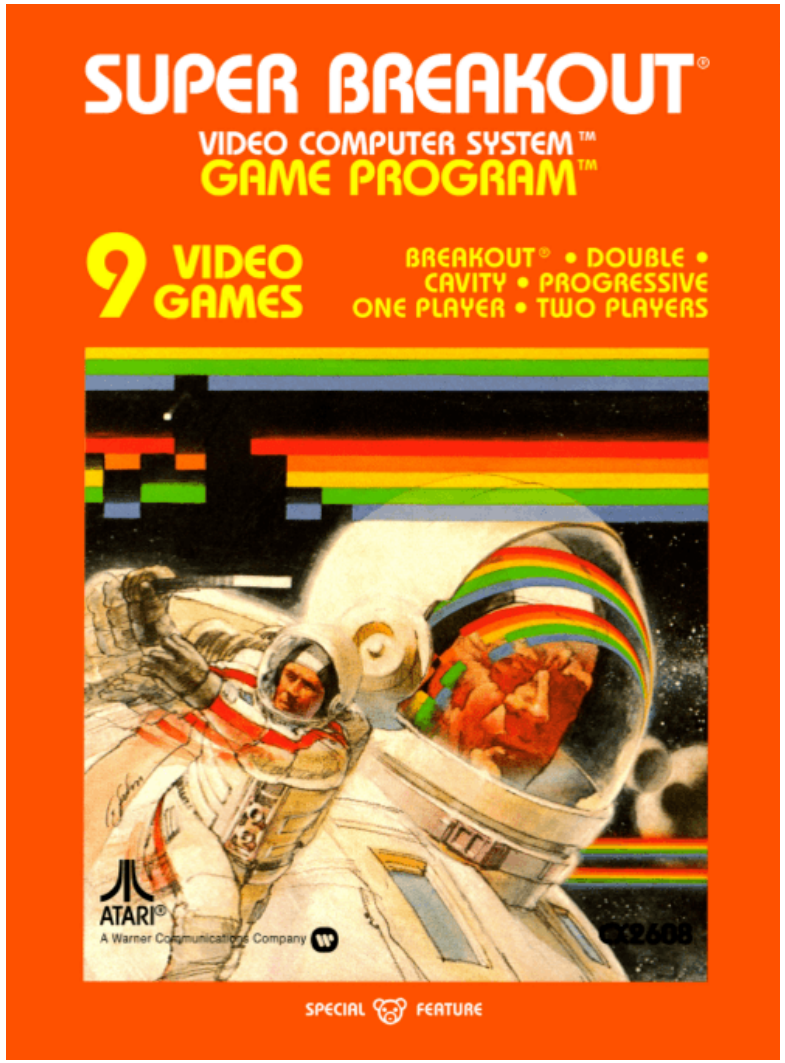
DQN: The Atari Benchmark



DQN: The Atari Benchmark



DQN: The Atari Benchmark



DQN: Problems with Naive Approach

DQN: Problems with Naive Approach

Candidate Algorithm:

loop

$$a \leftarrow \text{argmax}\, Q(s,a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$$

$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

$$s \leftarrow s'$$

DQN: Problems with Naive Approach

Candidate Algorithm:

loop

$$a \leftarrow \text{argmax}\, Q(s,a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$$

$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

$$s \leftarrow s'$$

Problems:

1. Samples Highly Correlated

DQN: Problems with Naive Approach

Candidate Algorithm:

loop

$$a \leftarrow \text{argmax}\, Q(s,a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$$

$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

$$s \leftarrow s'$$

Problems:

1. Samples Highly Correlated
2. Size-1 batches

DQN: Problems with Naive Approach

Candidate Algorithm:

loop

$$a \leftarrow \text{argmax}\, Q(s,a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$$

$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

$$s \leftarrow s'$$

Problems:

1. Samples Highly Correlated
2. Size-1 batches
3. Moving target

DQN: Problems with Naive Approach

Candidate Algorithm:

loop

$$a \leftarrow \text{argmax}\, Q(s,a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$$

$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right)^2$$

$$s \leftarrow s'$$

Problems:

1. Samples Highly Correlated
2. Size-1 batches
3. Moving target

} data buffer / experience replay

} periodically freeze target

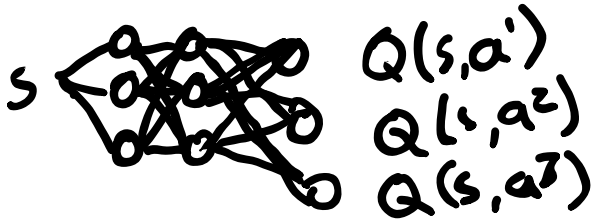
DQN

DQN

Q Network Structure:

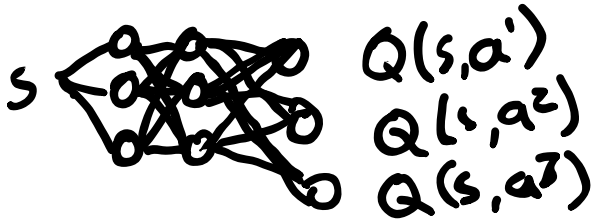
DQN

Q Network Structure:



DQN

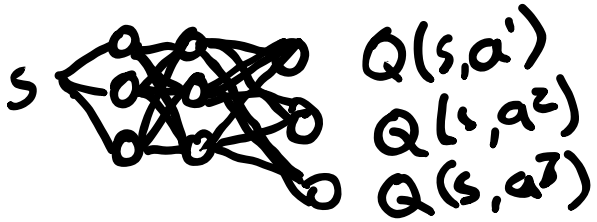
Q Network Structure:



Experience Tuple: (s, a, r, s')

DQN

Q Network Structure:

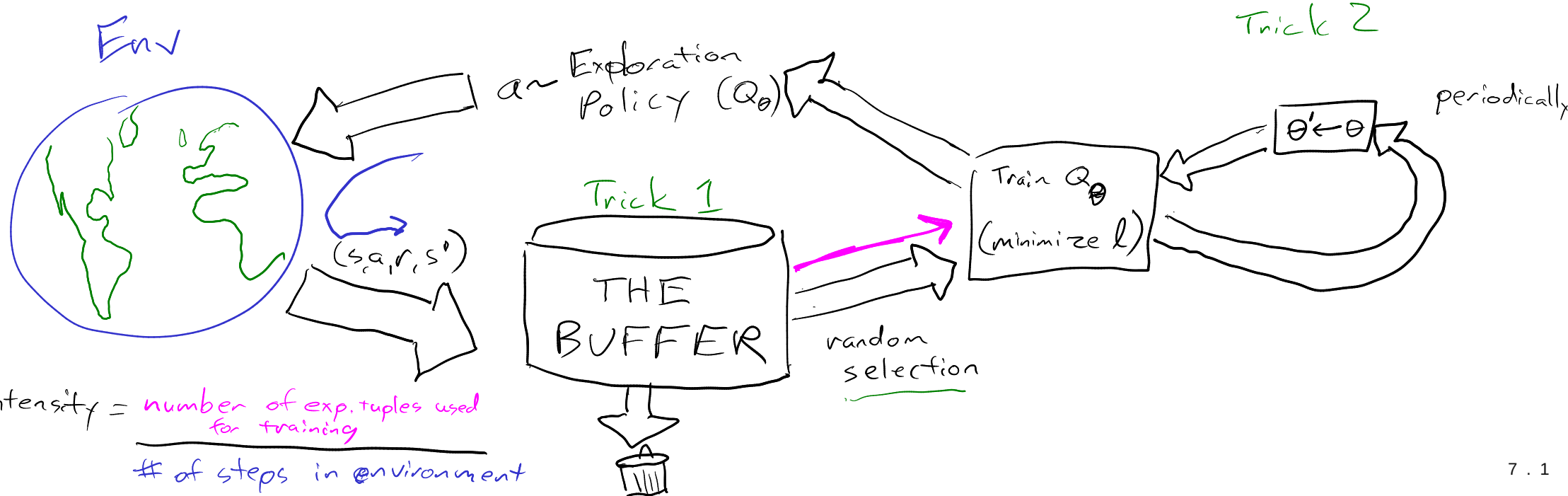


Experience Tuple: (s, a, r, s')

Loss:

$$l(s,a,r,s') = \left(r + \gamma \max_{a'} Q_{\theta'}(s',a') - Q_{\theta}(s,a)\right)^2$$

Double Q: $Q_{\theta_1}(s', \text{argmax}_{a'} Q_{\theta_2}(s',a'))$



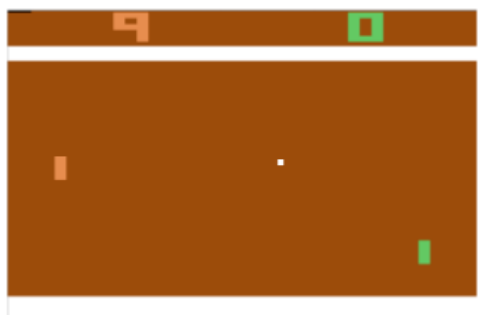
Training Intensity = $\frac{\text{number of exp. tuples used for training}}{\text{\# of steps in environment}}$

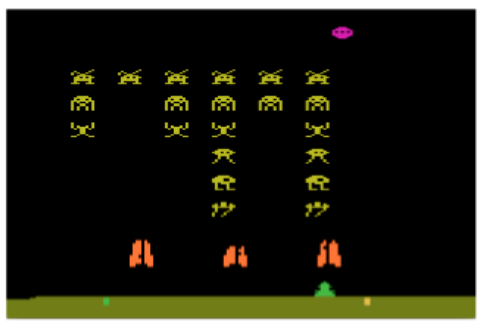
Playing Atari with Deep Reinforcement Learning

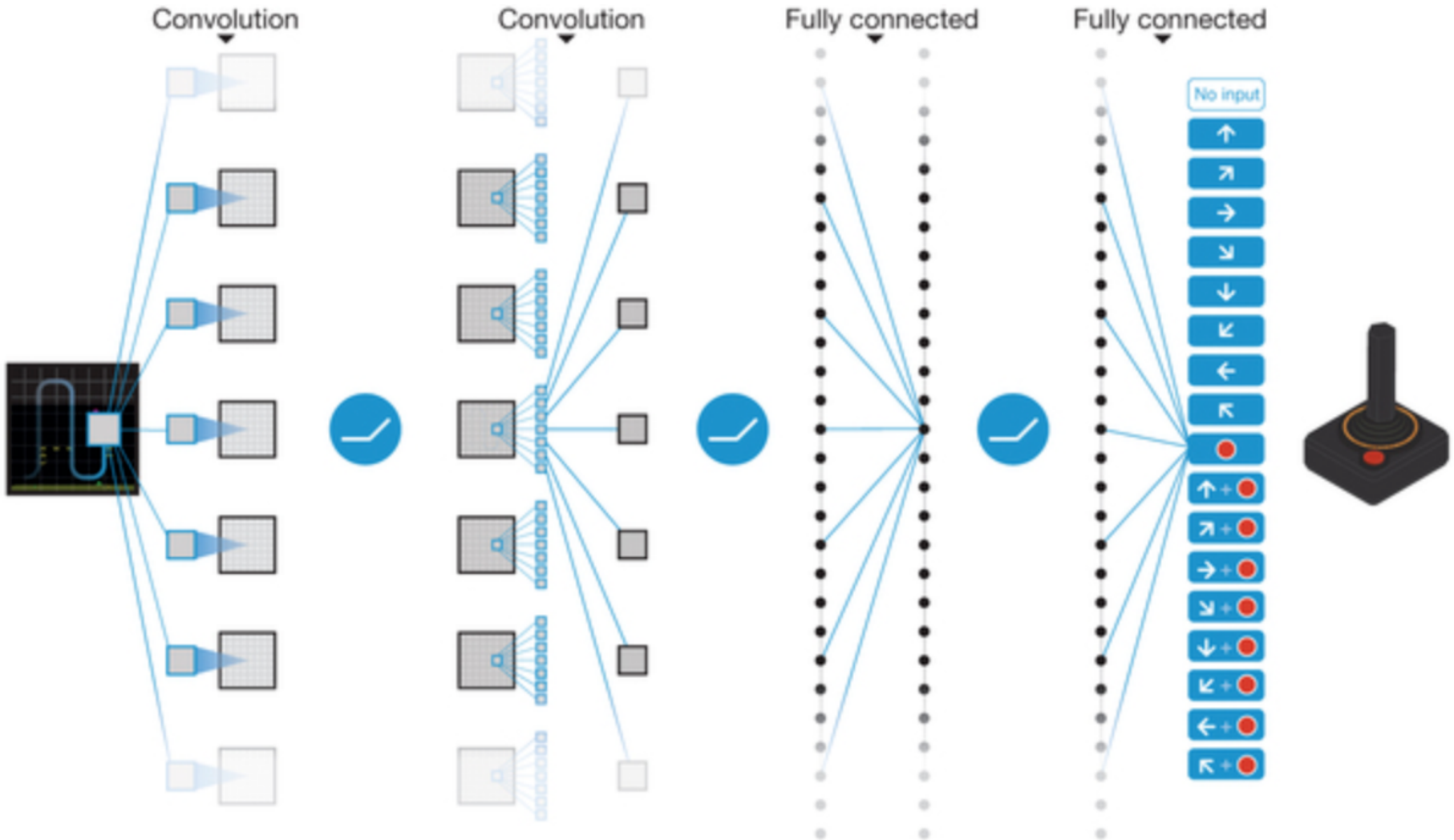
Volodymyr Mnih **Koray Kavukcuoglu** **David Silver** **Alex Graves** **Ioannis Antonoglou**

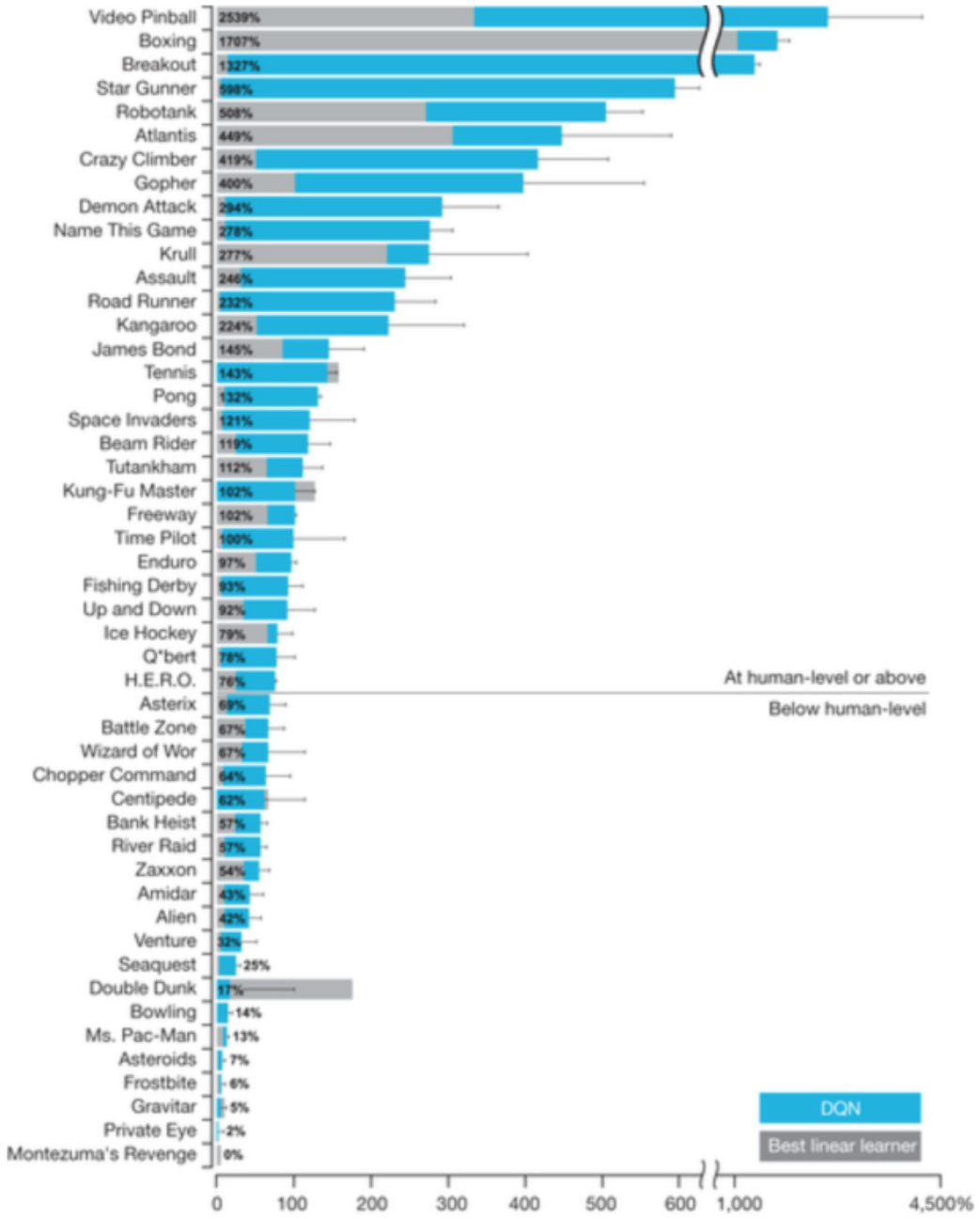
Daan Wierstra **Martin Riedmiller**

DeepMind Technologies

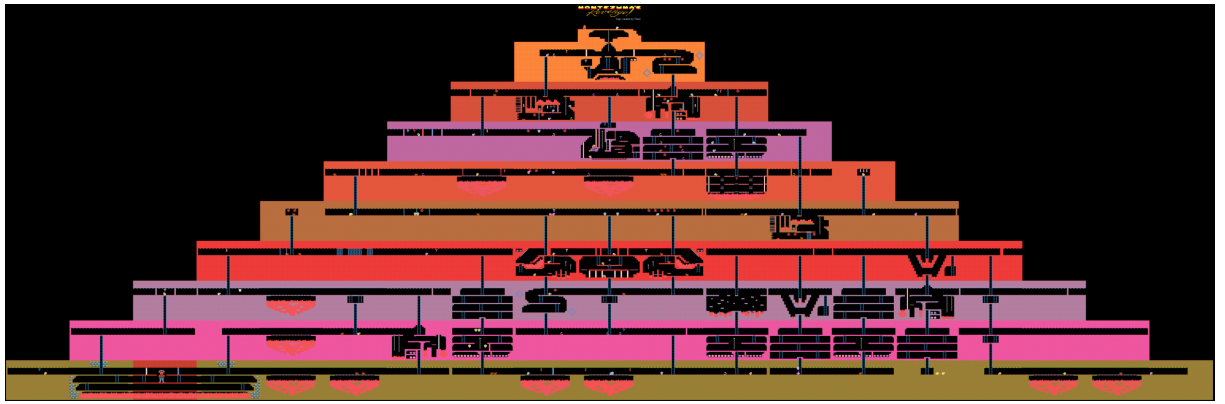






Convolution
Convolution
Fully connected
Fully connected
No input



https://www.youtube.com/watch?v=SuZVyOlgVek



Rainbow

Rainbow

- Double Q Learning

Rainbow

- Double Q Learning
- Prioritized Replay
  (priority proportional to last TD error)

Rainbow

- Double Q Learning
- Prioritized Replay
  (priority proportional to last TD error)
- Dueling networks
  Value network + advantage network
  $Q(s, a) = V(s) + A(s, a)$

Rainbow

- Double Q Learning
- Prioritized Replay
  (priority proportional to last TD error)
- Dueling networks
  Value network + advantage network
  $Q(s,a) = V(s) + A(s,a)$
- Multi-step learning
  $(r_t + \gamma r_{t+1} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma \max Q_\theta(s_{t+n}, a') - Q_\theta(s_t, a_t))^2$

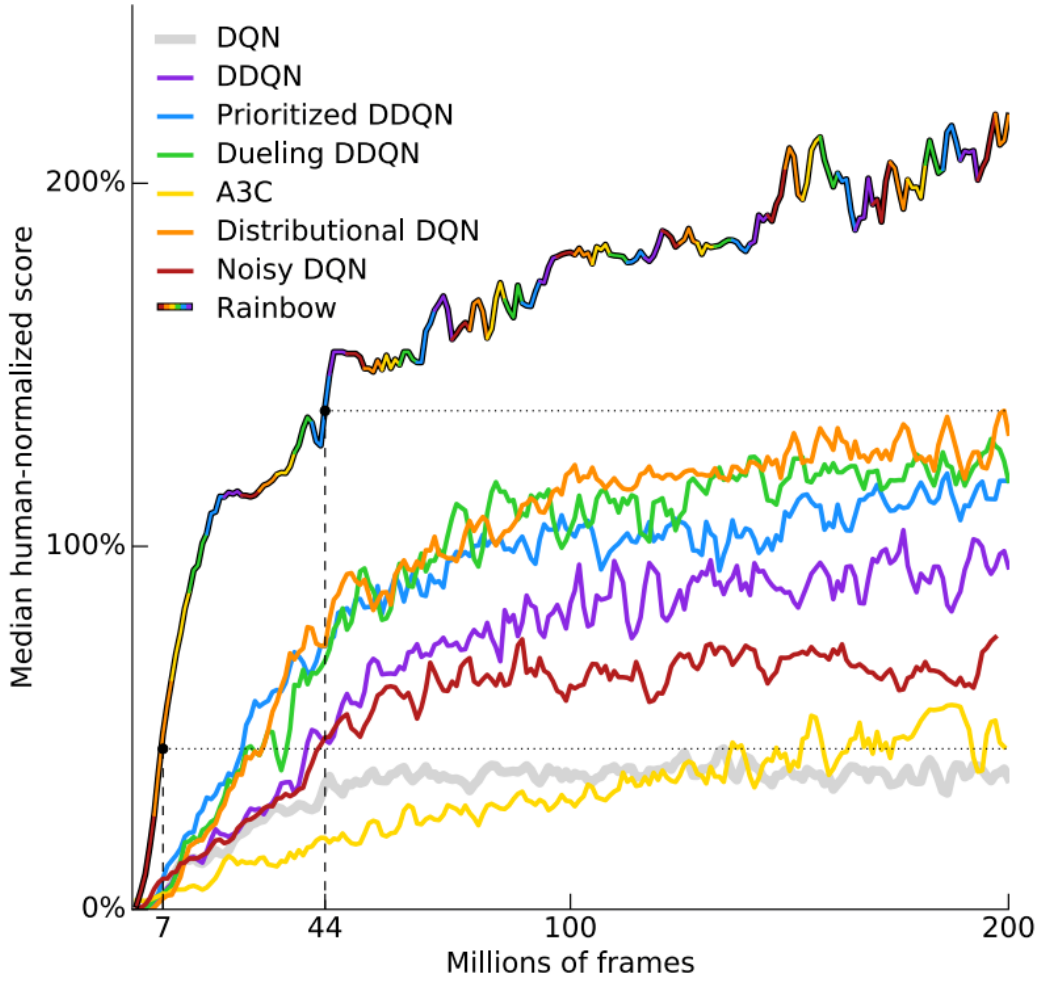
Rainbow

- Double Q Learning
- Prioritized Replay
  (priority proportional to last TD error)
- Dueling networks
  Value network + advantage network
  $Q(s,a) = V(s) + A(s,a)$
- Multi-step learning
  $(r_t + \gamma r_{t+1} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma \max Q_\theta(s_{t+n}, a') - Q_\theta(s_t, a_t))^2$
- Distributional RL
  predict an entire distribution of values instead of just Q

Rainbow

- Double Q Learning
- Prioritized Replay
  (priority proportional to last TD error)
- Dueling networks
  Value network + advantage network
  $Q(s,a) = V(s) + A(s,a)$
- Multi-step learning
  $(r_t + \gamma r_{t+1} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma \max Q_\theta(s_{t+n}, a') - Q_\theta(s_t, a_t))^2$
- Distributional RL
  predict an entire distribution of values instead of just Q
- Noisy Nets

Rainbow



- Double Q Learning
- Prioritized Replay
  (priority proportional to last TD error)
- Dueling networks
  Value network + advantage network
  $Q(s,a) = V(s) + A(s,a)$
- Multi-step learning
  $(r_t + \gamma r_{t+1} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma \max Q_\theta(s_{t+n}, a') - Q_\theta(s_t, a_t))^2$
- Distributional RL
  predict an entire distribution of values instead of just Q
- Noisy Nets

Actual Learning Curves

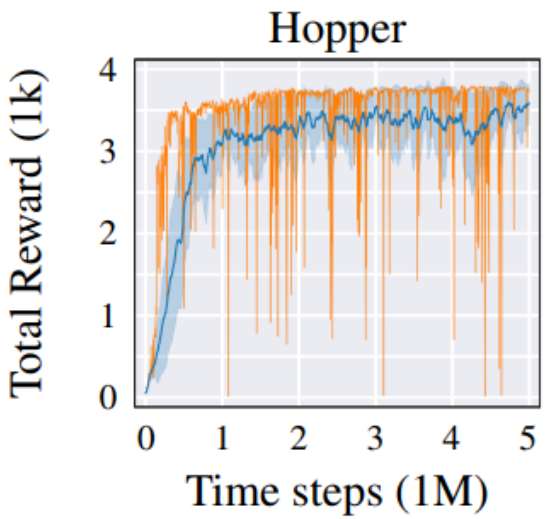


Figure 24: Standard presentation Single seed
The training curve as commonly presented in RL papers and the training curve of a single seed . Both curves are from the TD3 algorithm, trained for 5M time steps. The standard presentation is to evaluate every N_{freq} steps, average scores over N_{episodes} evaluation episodes and N_{seeds} seeds, then to smooth the curve by averaging over a window of N_{window} evaluations. (In our case this corresponds to $N_{\text{freq}} = 5000$, $N_{\text{episodes}} = 10$, $N_{\text{seeds}} = 10$, $N_{\text{window}} = 10$). The learning curve of a single seed has no smoothing over seeds or evaluations ($N_{\text{seeds}} = 1$, $N_{\text{window}} = 1$). By averaging over many seeds and evaluations, the training curves in RL can appear deceptively smooth and stable.

Paper: For SALE: State-Action Representation Learning for Deep Reinforcement Learning

Part II
Improved Policy Gradients

Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$$\uparrow$$

$$\hat{U}^\pi(s_k)$$

Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$$\uparrow \qquad\qquad \uparrow$$

$$\hat{Q}^\pi(s_k, a_k) \qquad \hat{U}^\pi(s_k)$$

Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$$\uparrow \qquad\qquad \uparrow$$

$$\hat{Q}^\pi(s_k, a_k) \qquad \hat{U}^\pi(s_k)$$

Advantage Function:

$$A(s,a) \equiv Q(s,a) - U(s)$$

$$A^*(s,a) = Q^*(s,a) - U^*(s)$$

$$A^\pi(s,a) = Q^\pi(s,a) - U^\pi(s)$$

Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$$\uparrow \qquad \uparrow$$

$$\hat{Q}^\pi(s_k, a_k) \qquad \hat{U}^\pi(s_k)$$

Advantage Function:

$A(s, a) \equiv Q(s, a) - U(s)$

loop:

$\tau \leftarrow \text{simulate\_episode}(\pi_\theta)$

$\widehat{\nabla_\theta U(\theta)} \leftarrow \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k \hat{A}_k$

$\theta \leftarrow \theta + \alpha \widehat{\nabla_\theta U(\theta)}$

Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$$\uparrow \qquad \uparrow$$

$$\hat{Q}^\pi(s_k, a_k) \qquad \hat{U}^\pi(s_k)$$

Advantage Function:

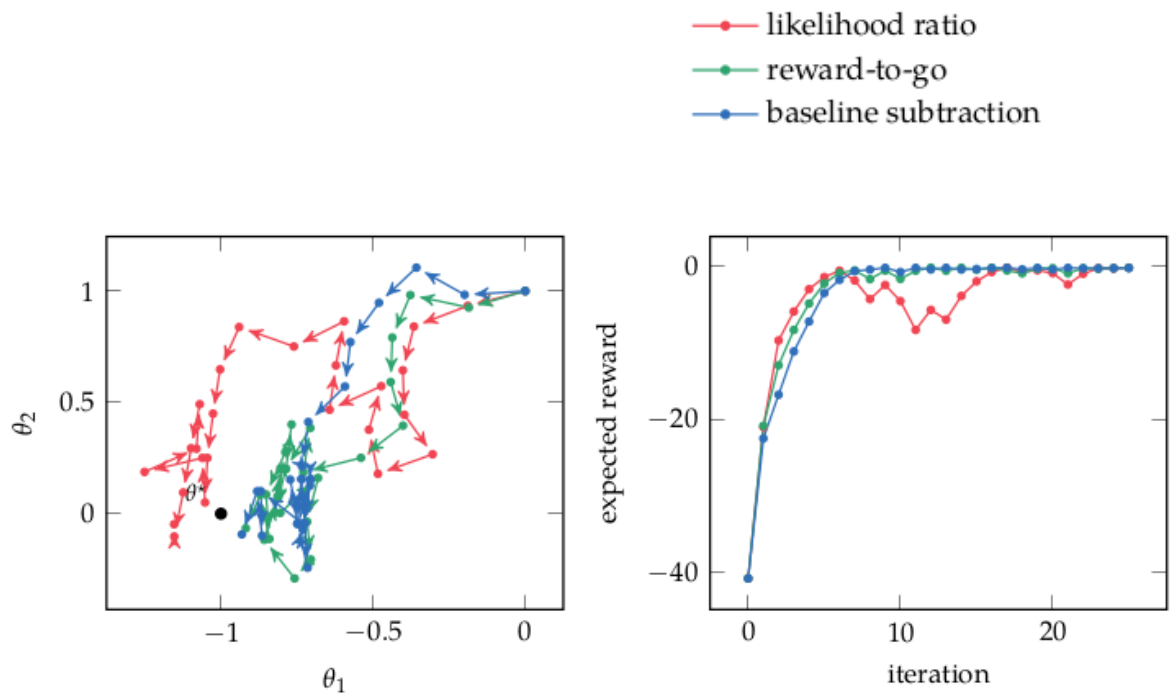
$$A(s,a) \equiv Q(s,a) - U(s)$$

loop:

$\tau \leftarrow \text{simulate\_episode}(\pi_\theta)$

$\widehat{\nabla_\theta U(\theta)} \leftarrow \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k \hat{A}_k$

$\theta \leftarrow \theta + \alpha \widehat{\nabla_\theta U(\theta)}$



Policy Gradient

$$\widehat{\nabla_\theta U(\theta)} = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$$\uparrow \qquad \uparrow$$

$$\hat{Q}^\pi(s_k, a_k) \qquad \hat{U}^\pi(s_k)$$

Advantage Function:

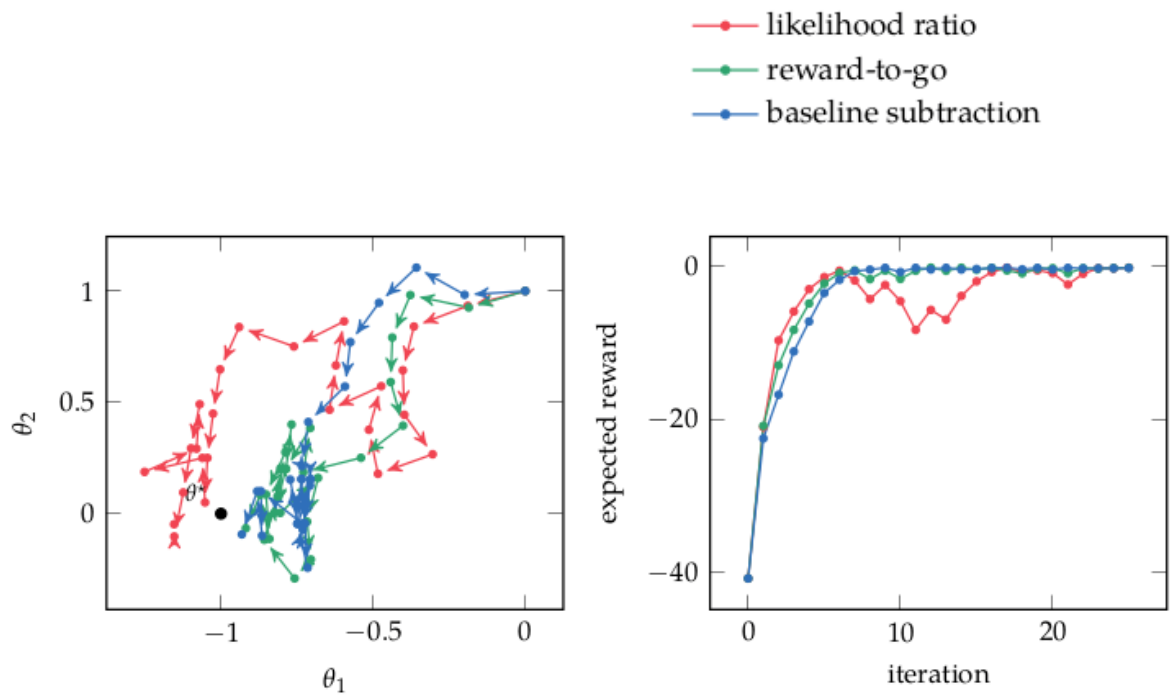
$$A(s,a) \equiv Q(s,a) - U(s)$$

loop:

$\tau \leftarrow \text{simulate\_episode}(\pi_\theta)$

$\widehat{\nabla_\theta U(\theta)} \leftarrow \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k \hat{A}_k$

$\theta \leftarrow \theta + \alpha \widehat{\nabla_\theta U(\theta)}$



Some problems:

- $\widehat{\nabla_\theta U(\theta)}$ has high variance
- The gradient is not a good step direction
- Fixed α steps are too big or small

Trust Region Optimization

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid \boldsymbol{s}_k) \gamma^k \, \hat{A}_k$$

$$\theta' = \theta + \alpha \widehat{\nabla U}(\theta)$$

Trust Region Optimization

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid \boldsymbol{s}_k) \gamma^k \hat{A}_k$$

$$\theta' = \theta + \alpha \widehat{\nabla U}(\theta)$$

$$\begin{aligned} \underset{\theta'}{\text{maximize}} \quad & U(\theta') \approx U(\theta) + \nabla U(\theta)^\top (\theta' - \theta) \\ \text{subject to} \quad & g(\theta, \theta') \leq \epsilon \end{aligned}$$

Trust Region Optimization

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k \hat{A}_k$$

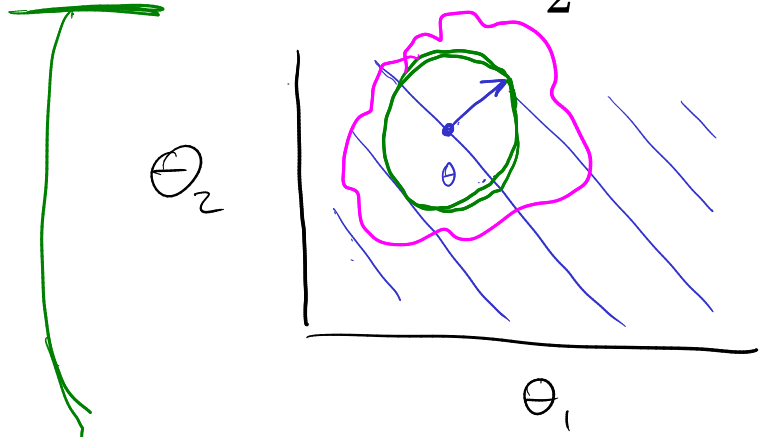
$$\theta' = \theta + \alpha \widehat{\nabla U}(\theta)$$

$$\underset{\theta'}{\text{maximize}} \qquad U(\theta') \approx U(\theta) + \nabla U(\theta)^\top (\theta' - \theta)$$

$$\text{subject to} \qquad g(\theta, \theta') \leq \epsilon$$

Example:

$$g(\theta, \theta') = \|\theta - \theta'\|_2^2 = \frac{1}{2}(\theta' - \theta)^\top (\theta' - \theta)$$



$$\theta' = \theta + \mathbf{u}\sqrt{\frac{2\epsilon}{\mathbf{u}^\top \mathbf{u}}} = \theta + \sqrt{2\epsilon}\frac{\mathbf{u}}{\|\mathbf{u}\|}$$

$$\mathbf{u} = \nabla U(\theta)$$

Natural Gradient

$$g(\theta, \theta') = D_{\mathrm{KL}}\left(p(\cdot \mid \theta) \,\middle|\middle|\, p(\cdot \mid \theta')\right) \leq \epsilon$$

Natural Gradient

$$g(\theta, \theta') = D_{\mathrm{KL}}\left(p(\cdot \mid \theta) \,\middle|\middle|\, p(\cdot \mid \theta')\right) \leq \epsilon$$

$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

Natural Gradient

$$g(\theta, \theta') = D_{\mathrm{KL}}\big(p(\cdot \mid \theta) \,\big|\big|\, p(\cdot \mid \theta')\big) \leq \epsilon$$

$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^{\top} \mathbf{F}_{\theta} (\theta' - \theta) \leq \epsilon$$

Natural Gradient

$$g(\theta, \theta') = D_{\mathrm{KL}}\big(p(\cdot \mid \theta) \,\big|\big|\, p(\cdot \mid \theta')\big) \leq \epsilon$$

$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon$$

↳ Fischer Information Matrix

$$\begin{aligned} \mathbf{F}_\theta &= \int p(\tau \mid \theta) \nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \, \mathrm{d}\tau \\ &= \mathbb{E}_\tau \Big[\nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \Big] \end{aligned}$$

Natural Gradient

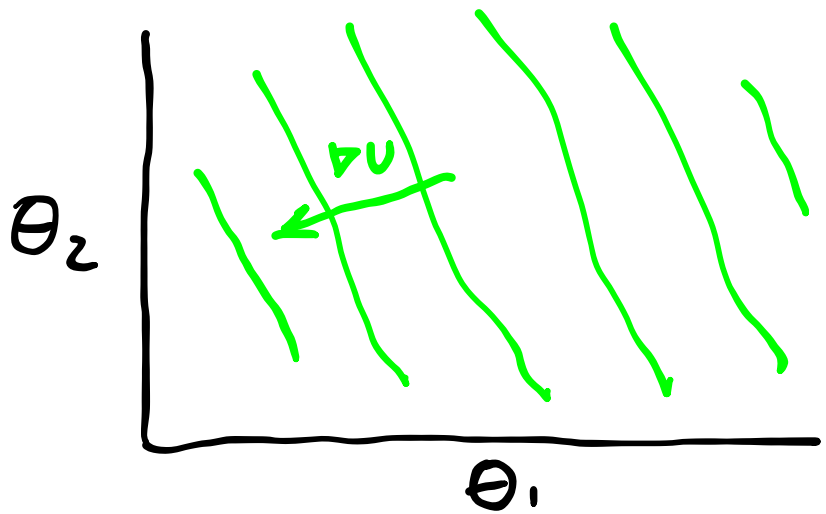
$$g(\theta, \theta') = D_{\mathrm{KL}}\left(p(\cdot \mid \theta) \,||\, p(\cdot \mid \theta')\right) \leq \epsilon$$

$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon$$

↳ Fischer Information Matrix

$$\mathbf{F}_\theta = \int p(\tau \mid \theta) \nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \, \mathrm{d}\tau$$
$$= \mathbb{E}_\tau \left[\nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \right]$$



Natural Gradient

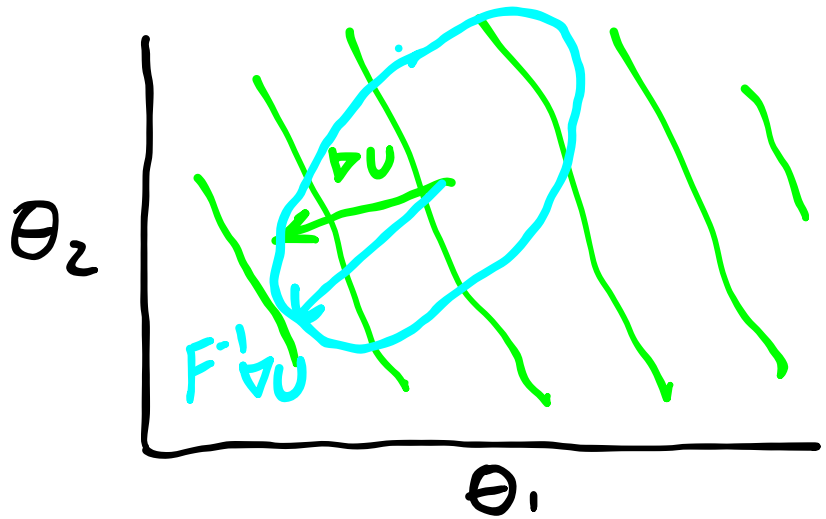
$$g(\theta, \theta') = D_{\mathrm{KL}}\left(p(\cdot \mid \theta) \,||\, p(\cdot \mid \theta')\right) \leq \epsilon$$

$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon$$

↳ Fischer Information Matrix

$$\mathbf{F}_\theta = \int p(\tau \mid \theta) \nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \, \mathrm{d}\tau$$
$$= \mathbb{E}_\tau \left[\nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \right]$$



Natural Gradient

$$g(\theta, \theta') = D_{\mathrm{KL}}\big(p(\cdot \mid \theta) \,\big|\big|\, p(\cdot \mid \theta')\big) \leq \epsilon$$

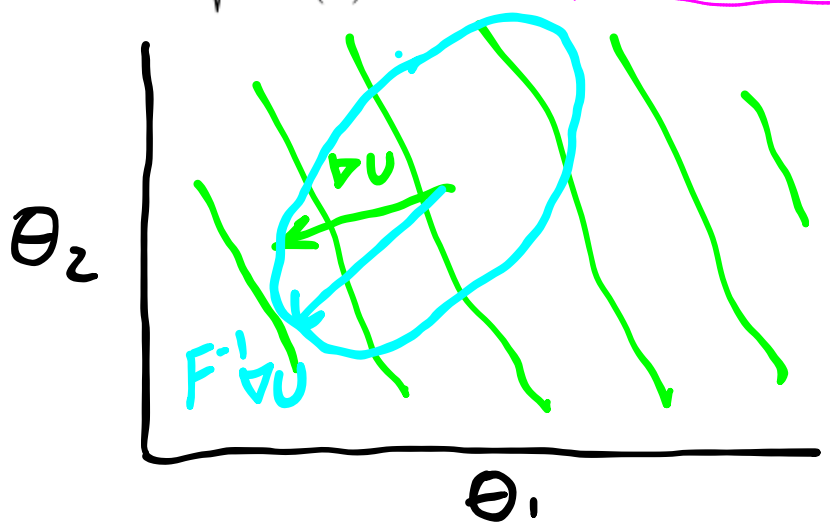
$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon$$

↳ Fischer Information Matrix

$$\mathbf{F}_\theta = \int p(\tau \mid \theta) \nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \, \mathrm{d}\tau$$
$$= \mathbb{E}_\tau \Big[\nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \Big]$$

$$\theta' = \theta + \mathbf{u} \sqrt{\frac{2\epsilon}{\nabla U(\theta)^\top \mathbf{u}}} \qquad \mathbf{u} = \mathbf{F}_\theta^{-1} \nabla U(\theta)$$



Natural Gradient

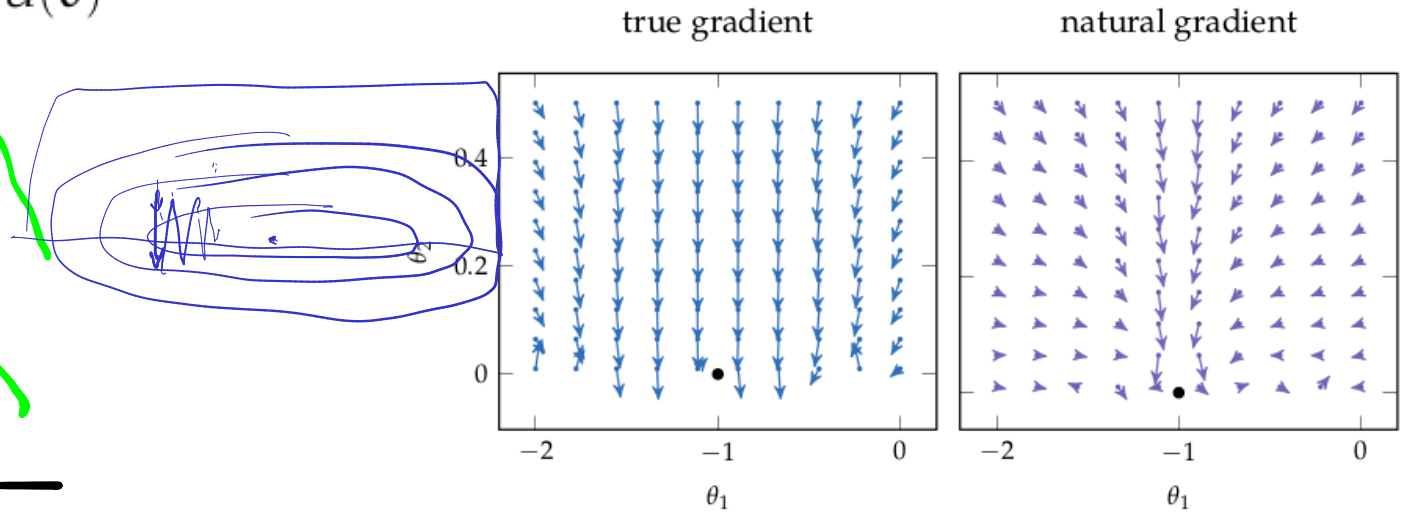
$$g(\theta, \theta') = D_{\mathrm{KL}}\big(p(\cdot \mid \theta) \,\big|\big|\, p(\cdot \mid \theta')\big) \leq \epsilon \qquad D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon$$

↳ Fischer Information Matrix

$$\mathbf{F}_\theta = \int p(\tau \mid \theta) \nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \mathrm{d}\tau$$
$$= \mathbb{E}_\tau \Big[\nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \Big]$$

$$\theta' = \theta + \mathbf{u} \sqrt{\frac{2\epsilon}{\nabla U(\theta)^\top \mathbf{u}}} \qquad \mathbf{u} = \mathbf{F}_\theta^{-1} \nabla U(\theta)$$



Conservative Policy Iteration Objective

Conservative Policy Iteration Objective

Discounted Visitation Distribution

Conservative Policy Iteration Objective

Discounted Visitation Distribution

$$\underset{\tau \sim \pi_\theta}{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right]$$

Conservative Policy Iteration Objective

Discounted Visitation Distribution

$$\underset{\tau \sim \pi_\theta}{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E}[R(s,a)]$$

Conservative Policy Iteration Objective

Discounted Visitation Distribution

$$\underset{\tau \sim \pi_\theta}{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E}[R(s,a)]$$

$$b_{\gamma,\theta}(s) \propto P(s_1 = s) + \gamma P(s_2 = s) + \gamma^2 P(s_3 = s) + \dots$$

Conservative Policy Iteration Objective

Discounted Visitation Distribution

$$\underset{\tau \sim \pi_\theta}{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E}[R(s,a)]$$

$$b_{\gamma,\theta}(s) \propto P(s_1 = s) + \gamma P(s_2 = s) + \gamma^2 P(s_3 = s) + \dots$$

Policy Advantage

$$f(\theta, \theta') = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_{\theta'}(\cdot|s)}}{E}\left[A^{\pi_\theta}(s,a)\right]$$

$$= \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E}\left[\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s,a)\right]$$

"Approximately Optimal Approximate RL"
Kakade & Langford

Trust Region Policy Optimization (TRPO)

Natural Gradient Approximation + Conservative Policy Iteration Objective

Update Step:

$$\underset{\theta'}{\text{maximize}} \quad f(\theta, \theta') = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E} \left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A^{\pi_\theta}(s, a) \right]$$

$$\text{subject to} \quad g(\theta, \theta') = \underset{s \sim b_{\gamma,\theta}}{\mathbb{E}} \left[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \parallel \pi_{\theta'}(\cdot \mid s)) \right] \leq \epsilon$$

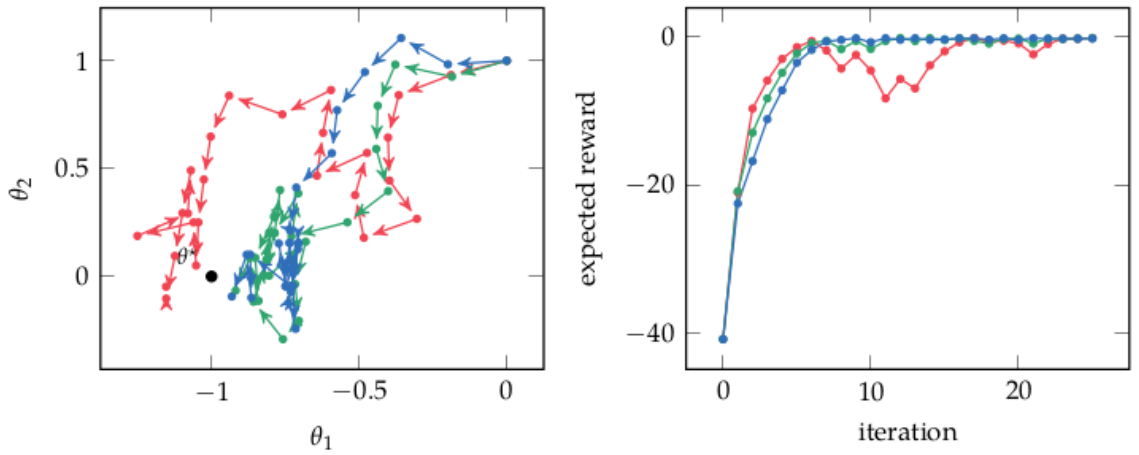
Trust Region Policy Optimization (TRPO)

Natural Gradient Approximation + Conservative Policy Iteration Objective

Update Step:

$$\underset{\theta'}{\text{maximize}} \quad f(\theta, \theta') = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E} \left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A^{\pi_\theta}(s, a) \right]$$

$$\text{subject to} \quad g(\theta, \theta') = \underset{s \sim b_{\gamma,\theta}}{\mathbb{E}} \left[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \pi_{\theta'}(\cdot \mid s)) \right] \leq \epsilon$$



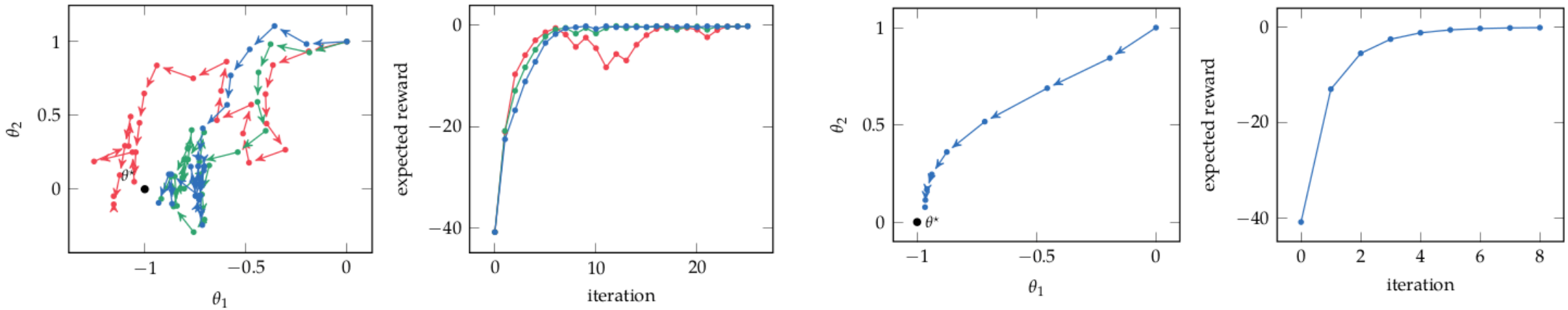
Trust Region Policy Optimization (TRPO)

Natural Gradient Approximation + Conservative Policy Iteration Objective

Update Step:

$$\underset{\theta'}{\text{maximize}} \quad f(\theta, \theta') = \underset{\substack{s \sim b_{\gamma,\theta} \\ a \sim \pi_\theta(\cdot|s)}}{E} \left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A^{\pi_\theta}(s, a) \right]$$

$$\text{subject to} \quad g(\theta, \theta') = \underset{s \sim b_{\gamma,\theta}}{\mathbb{E}} \left[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, \pi_{\theta'}(\cdot \mid s)) \right] \leq \epsilon$$



Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO)

Clipped Surrogate Objective:

Proximal Policy Optimization (PPO)

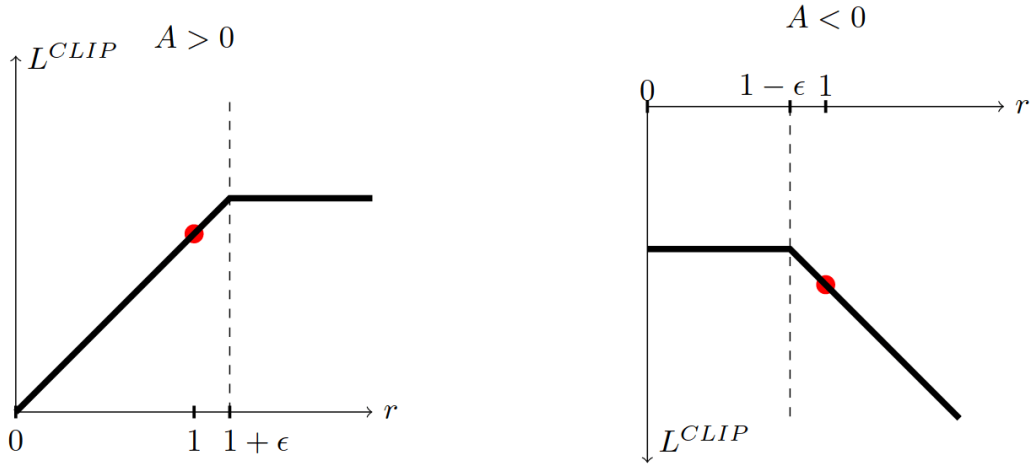
Clipped Surrogate Objective:

$$\underset{\substack{s\sim b_{\gamma,\theta}\\ a\sim\pi_\theta(\cdot|s)}}{E}\left[\min\left(\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)}A_\theta(s,a),\operatorname{clamp}\left(\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)},1-\epsilon,1+\epsilon\right)A_\theta(s,a)\right)\right]$$

Proximal Policy Optimization (PPO)

Clipped Surrogate Objective:

$$\underset{\substack{s\sim b_{\gamma,\theta}\\ a\sim\pi_\theta(\cdot|s)}}{E}\left[\min\left(\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)}A_\theta(s,a),\operatorname{clamp}\left(\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)},1-\epsilon,1+\epsilon\right)A_\theta(s,a)\right)\right]$$

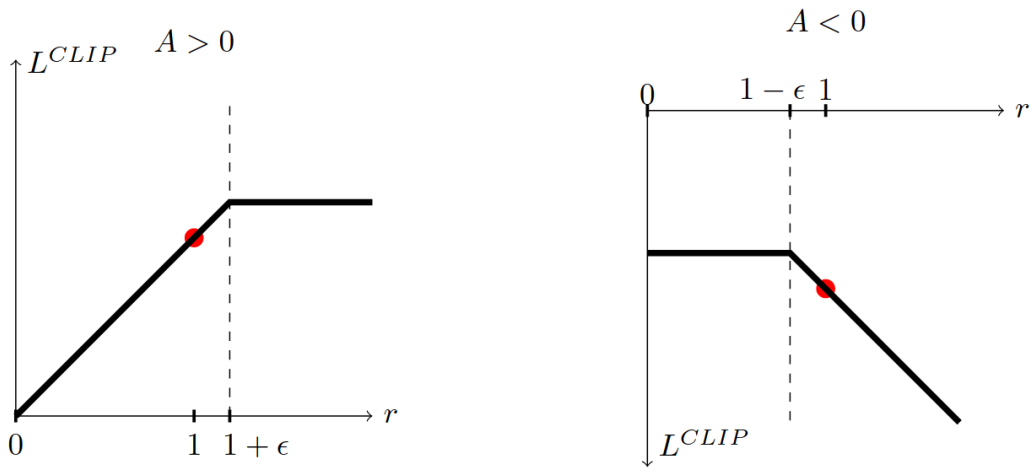


$$r=\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)}$$

Proximal Policy Optimization (PPO)

Clipped Surrogate Objective:

$$\underset{\substack{s\sim b_{\gamma,\theta}\\ a\sim\pi_\theta(\cdot|s)}}{E}\left[\min\left(\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)}A_\theta(s,a),\operatorname{clamp}\left(\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)},1-\epsilon,1+\epsilon\right)A_\theta(s,a)\right)\right]$$



$$r=\frac{\pi_{\theta'}(a\mid s)}{\pi_\theta(a\mid s)}$$

Each step: take several gradient steps with trajectories from π_θ

(No need for constraint because it will stop when it hits the clip)

PPO Impact

PPO Impact

Cited 29262 times (as of Fall 2025)

PPO Impact

Cited 29262 times (as Gall 2025)

(avg ~10x per day)

PPO Impact

Cited 29262 times (as Gall 2025)

(avg ~10x per day)

- John Schulman was a co-founder of OpenAI

PPO Impact

Cited 29262 times (as Gall 2025)

(avg ~10x per day)

- John Schulman was a co-founder of OpenAI
- InstructGPT (first large-scale RLHF) used PPO to get a GPT model to obey human intent (crucial to get a language model to act as a chatbot)

PPO Impact

Cited 29262 times (as Gall 2025)

(avg ~10x per day)

- John Schulman was a co-founder of OpenAI
- InstructGPT (first large-scale RLHF) used PPO to get a GPT model to obey human intent (crucial to get a language model to act as a chatbot)
- ChatGPT is a successor to InstructGPT

PPO Impact

Cited 29262 times (as Gall 2025)

(avg ~10x per day)

- John Schulman was a co-founder of OpenAI
- InstructGPT (first large-scale RLHF) used PPO to get a GPT model to obey human intent (crucial to get a language model to act as a chatbot)
- ChatGPT is a successor to InstructGPT

But...

Implementation Matters

Implementation Matters

Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry; ICLR 2020

Implementation Details

- Value function clipping
- Reward scaling
- Orthogonal initialization and layer scaling
- Adam learning rate annealing
- Reward clipping
- Observation normalization
- Observation clipping
- Hyperbolic tan activations
- Global gradient clipping

"PPO's marked improvement over TRPO (and even stochastic gradient descent) can be largely attributed to these optimizations."

Implementation Matters

Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry; ICLR 2020

Implementation Details

- Value function clipping
- Reward scaling
- Orthogonal initialization and layer scaling
- Adam learning rate annealing
- Reward clipping
- Observation normalization
- Observation clipping
- Hyperbolic tan activations
- Global gradient clipping

"PPO's marked improvement over TRPO (and even stochastic gradient descent) can be largely attributed to these optimizations."

Andrychowicz M, Raichuk A, Stańczyk P, et al. **What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study**. *arXiv*. Preprint posted online June 10, 2020:arXiv:2006.05990. doi:10.48550/arXiv.2006.05990

Implementation Matters

Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry; ICLR 2020

Implementation Details

- Value function clipping
- Reward scaling
- Orthogonal initialization and layer scaling
- Adam learning rate annealing
- Reward clipping
- Observation normalization
- Observation clipping
- Hyperbolic tan activations
- Global gradient clipping

"PPO's marked improvement over TRPO (and even stochastic gradient descent) can be largely attributed to these optimizations."

Andrychowicz M, Raichuk A, Stańczyk P, et al. **What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study**. *arXiv*. Preprint posted online June 10, 2020:arXiv:2006.05990. doi:10.48550/arXiv.2006.05990

The 37 Implementation Details of Proximal Policy Optimization

Part III
Actor-Critic

Actor-Critic

Actor-Critic

Which should we learn? A, Q, or V?

Actor-Critic

Which should we learn? A, Q, or V?

$$\nabla U(\theta) = E_\tau \left[\sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k))) \right]$$

Actor-Critic

Which should we learn? A, Q, or V?

$$\nabla U(\theta) = E_\tau \left[\sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k))) \right]$$

temporal difference residual

Actor-Critic

Which should we learn? A, Q, or V?

$$\nabla U(\theta) = E_\tau \left[\sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k))) \right]$$

temporal difference residual

$$l(\phi) = E\left[\left(V_\phi(s) - V^{\pi_\theta}(s) \right)^2 \right]$$

Actor-Critic

Which should we learn? A, Q, or V?

$$\nabla U(\theta) = E_\tau \left[\sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k (r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k))) \right]$$

temporal difference residual

$$l(\phi) = E\left[\left(V_\phi(s) - V^{\pi_\theta}(s) \right)^2 \right]$$

↑ estimate with reward to go from sims

Generalized Advantage Estimation

Recap

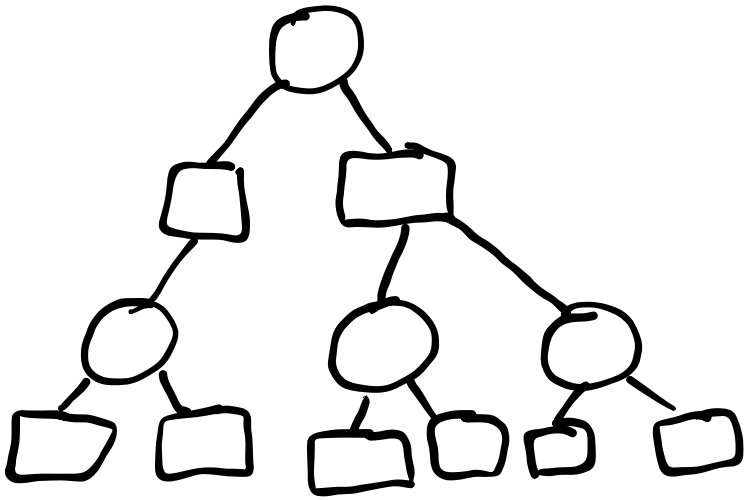
Alpha Zero: Actor Critic with MCTS

Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS

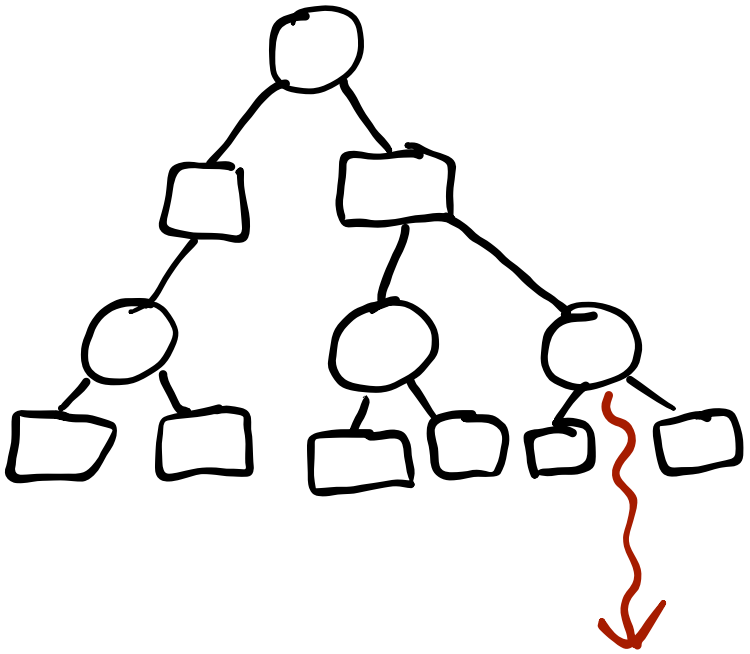
Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS



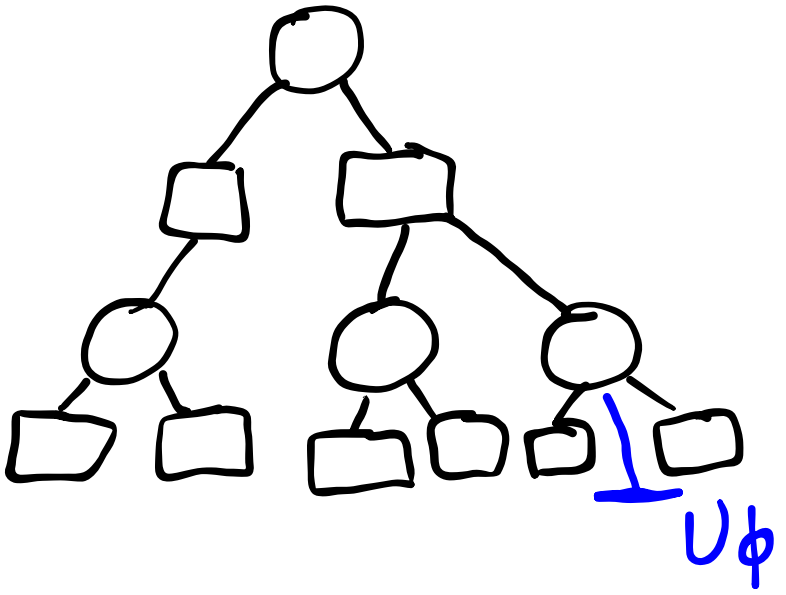
Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS



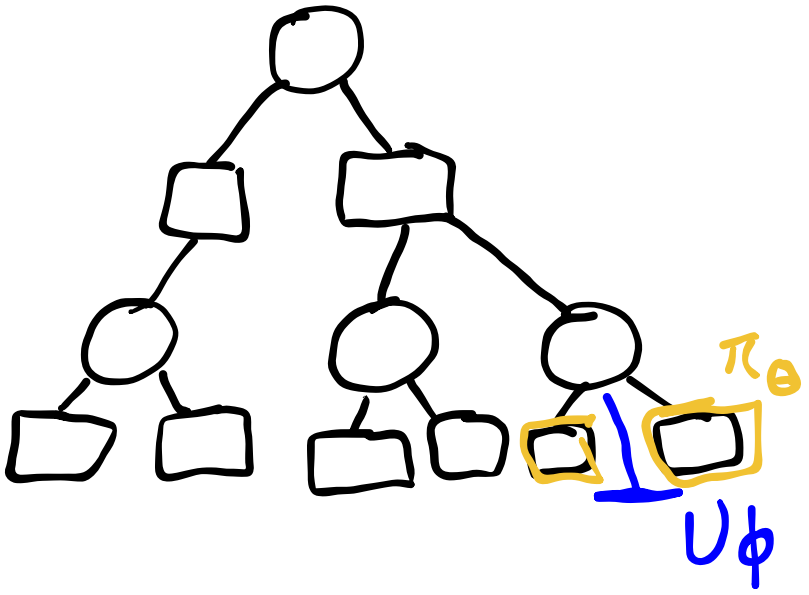
Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS



Alpha Zero: Actor Critic with MCTS

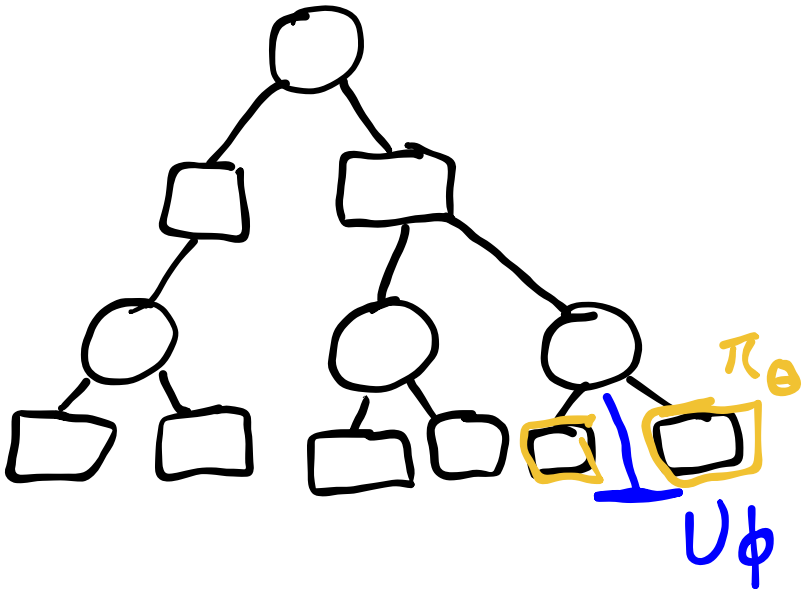
1. Use π_θ and U_ϕ in MCTS



$$a = \arg\max_a Q(s,a) + c\pi_\theta(a \mid s)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS
2. Learn π_θ and U_ϕ from tree



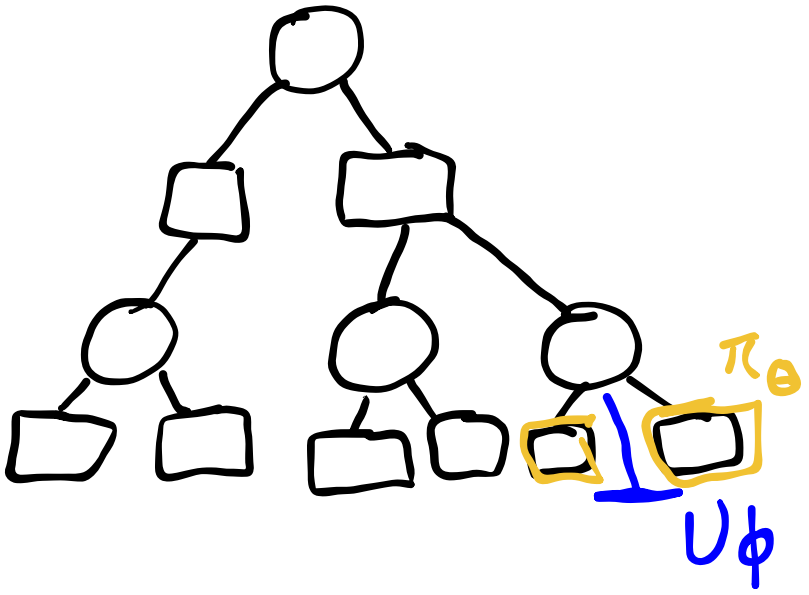
$$a = \arg\max_a Q(s,a) + c\pi_\theta(a \mid s)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS
2. Learn π_θ and U_ϕ from tree

$$\ell(\theta) = -\mathbb{E}_s\left[\sum_a \pi_{\text{MCTS}}(a \mid s) \log \pi_\theta(a \mid s)\right]$$

$$\pi_{\text{MCTS}}(a \mid s) \propto N(s,a)^\eta$$



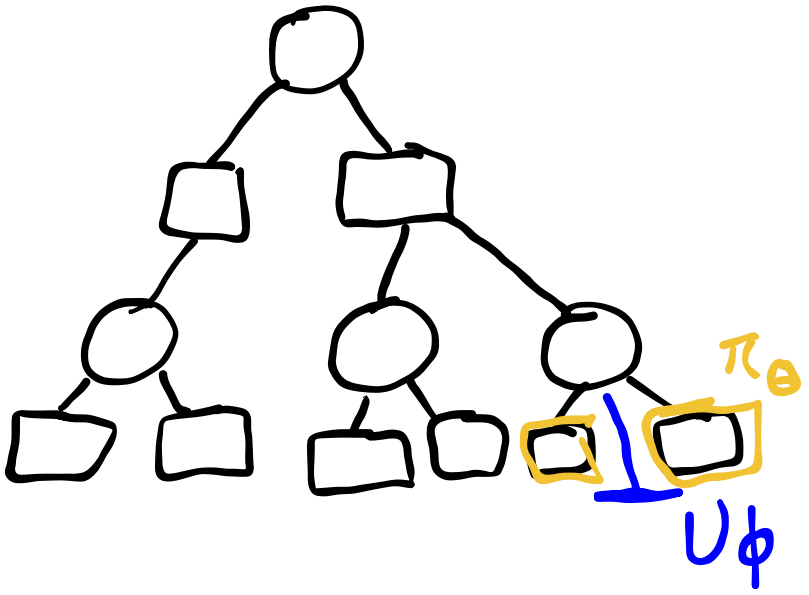
$$a = \arg\max_a Q(s,a) + c\pi_\theta(a \mid s)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS
2. Learn π_θ and U_ϕ from tree

$$\ell(\theta) = -\mathbb{E}_s\left[\sum_a \pi_{\text{MCTS}}(a \mid s) \log \pi_\theta(a \mid s)\right]$$

$$\pi_{\text{MCTS}}(a \mid s) \propto N(s,a)^\eta$$



$$\ell(\phi) = \frac{1}{2}\mathbb{E}_s\left[\left(U_\phi(s) - U_{\text{MCTS}}(s)\right)^2\right]$$

$$U_{\text{MCTS}}(s) = \max_a Q(s,a)$$

$$a = \arg\max_a Q(s,a) + c\pi_\theta(a \mid s)\frac{\sqrt{N(s)}}{1+N(s,a)}$$